Debre Berhan University

College of Computing
Department of Computer Science

Internship Report

Project Tittle: Development of a Component-Based Push Notification System(PNS)

Hosting company: AFRICOM TECHNOLOGIES PLC

INTERNEES: BINIYAM TEHAKELE(DBU1501070)

ACADEMIC MENTOR: SIRAGE Z.

COMPANY SUPERVISOR: ROBEL

September 2025

Africom Techologies PLC,Addis Ababa Ethiopia

PART ONE: INTRODUCTION

Declaration

I, the undersigned, declare that this internship report entitled *"Push Notification System (PNS)"*, prepared in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science at Debre Berhan University, is my original work carried out at Africom Technologies PLC during my internship period.
To the best of my knowledge, this report has not been presented, in whole or in part, to any other institution for the award of a degree or diploma.

Name	ID	Signature	Date
Biniyam Tehakele	DBU1501070		October 2018 E.C

Name of the Advisor	Signiture	Date
_____	_____	_____

Acknowledgements

First and foremost, I would like to express my gratitude to the Almighty God for granting me the strength, health, and perseverance to successfully complete my internship.
I am sincerely thankful to Africom Technologies PLC for providing me with the opportunity to undertake my internship in their organization. The professional environment, resources, and mentorship I received were invaluable to my growth.

My deepest appreciation goes to my university mentor, Ms.Sirage z., for his continuous guidance, advice, and constructive feedback throughout the course of this work. I am equally grateful to my company supervisor, Mr.Robel, whose patience and support enabled me to learn and apply practical skills effectively.

Lastly, I extend my appreciation to my colleagues, classmates, friends, and family for their encouragement and support during this journey.

An Executive Summary

This report presents the internship experience I undertook at Africom Technologies PLC as part of the requirements for the Bachelor of Science degree in Computer Science at Debre Berhan University.

During my internship, I was engaged in the design and development of a Push Notification System (PNS) using ASP.NET(framework), C#(programming language, SQL Server Management Studio(Database, and SMTP integration, with Swagger UI as the testing and interface platform. The project aimed to create a reliable system that enables organizations to send notifications to their employees or customers efficiently.

The report is structured into five main parts:

- Part One includes preliminary sections such as the cover page, declaration, acknowledgements, executive summary, table of contents, and list of tables/figures.
- Part Two discusses the background of Africom Technologies, including its history, services, customers, and workflows.
- Part Three details my internship experience, the tasks I performed, tools used, challenges faced, and the skills gained.
- Part Four presents the project work, including the problem statement, objectives, methodology, system design, results, and screenshots.
- Part Five concludes with overall findings, recommendations, references, and appendices.

The internship enabled me to apply my academic knowledge in a real-world setting, enhance my technical skills, and strengthen my teamwork, communication, and leadership abilities. The Push Notification System (PNS) project provided me with hands-on experience in system development, database management, and software testing, preparing me for future professional challenges.

Table of Contents

List of Tables and Figures

List Of Table

List Of Figure

Part Two: Company Background

2.1 Company Short History and Profile

Africom Technologies PLC is a highly prominent, indigenous Ethiopian **Information Technology (IT) solutions and services company**, playing a leading role in the country's digital transformation efforts. The company was officially established in **2004 G.C. (1996 E.C.)** by local entrepreneurs with a vision to deliver world-class IT services within Ethiopia and beyond.

The company's initial focus was modest, centered on basic **ICT training** and providing services for **small-scale website development**. However, Africom quickly demonstrated an ability to handle increasingly large-scale and complex enterprise solutions. Its dedication to quality and professional execution propelled its rapid expansion across key public and private sectors.

Africom's enduring commitment to adopting and adhering to international best practices is evident in its attainment of **ISO certification**, which has been instrumental in establishing its reputation for reliable, high-quality service delivery. Over two decades of operation, the company has successfully completed **over 50 major projects** for a client base exceeding **250 organizations**, firmly reinforcing its status as a cornerstone firm in Ethiopia's expanding digital economy.

The company's strategic location within the **Ethiopian ICT Park in Addis Ababa** positions it at the center of the nation's technology hub, fostering collaboration and innovation.

- **Strategic Vision:** To become the most preferred IT solution and service provider in Africa by 2030, reflecting a commitment to regional expansion and technical excellence.
- **Location:** Ethiopian ICT Park, Addis Ababa, Ethiopia.

2.2 Main Products, Services, and Technologies (HW & SW)

Africom Technologies provides a comprehensive and integrated suite of services designed to enable **digital transformation** across various industries. The company supports the complete lifecycle of enterprise IT needs, from initial strategic consultancy to ongoing maintenance.

2.2.1. Core Service Offerings

Service Category	Description	Primary Focus / Examples
Custom Software Development	Delivery of scalable, high-performance enterprise, web, and mobile applications tailored to specific client needs.	ERP systems, E-commerce Platforms, and Single Window Solutions for government services.
Business Process Outsourcing (BPO)	Provision of offshore, onshore, and nearshore services, handling routine, IT-enabled business tasks.	International BPO for European and Asian clients, data processing, and IT helpdesk management.

Service Category	Description	Primary Focus / Examples
IT Consultancy and Analysis	Offering strategic ICT planning, ensuring technology roadmaps are tightly aligned with core business objectives.	IT Strategy Development, Business Requirements Analysis, and system feasibility studies.
Quality Assurance (QA) & IT Audit	Independent assessment of deployed systems and infrastructure for optimal efficiency, security, and regulatory compliance.	System Testing, Vulnerability Assessment, and pre-deployment security audits.
Training and Capacity Building	Providing accredited and customized IT training programs aimed at upskilling client staff.	Skill Transfer during and after implementation, specialized training on new platforms.
Support and Maintenance	Comprehensive technical support under formal Service Level Agreements (SLAs) to maintain IT infrastructure and application health.	Managed IT Services, Helpdesk Support, and proactive system monitoring.

Table 2.1 – Core Service Offerings

2.2.2. Key Technologies Utilized

- **Software (SW)**: The primary technology for developing scalable Web APIs and robust enterprise backends is C# and the ASP.NET Core framework (used in the Notification System project). Frontend development leverages modern JavaScript frameworks such as React and Angular. For data persistence, Microsoft SQL Server is utilized as the primary relational database management system.
- **Development Tools:** Git/GitHub is strictly enforced for version control. Visual Studio serves as the standard integrated development environment (IDE).
- **Hardware & Infrastructure (HW**): Africom utilizes secured high-capacity internal servers and leverages major cloud platforms (such as Microsoft Azure and Amazon Web Services (AWS)) to guarantee high availability and scalability for its clients.

2.3 Main Customers / End Users

Africom serves a highly diverse and demanding client base, reflecting its capability to handle complex governmental projects and meet stringent international outsourcing requirements.

- **Government & Public Institutions:** This segment involves large-scale **digitization projects**, the development of public **web portals**, and complex **data management systems** for federal, regional, and city administrations.

- **Financial & Telecom Sector:** Africom services major **banks**, **telecom companies**, and emerging **fintech firms**, providing secure custom software and system integration, where reliability and regulatory compliance are paramount.
- **International Clients (BPO):** Through its Business Process Outsourcing division, Africom supports companies in international markets, primarily in **Europe** and **Asia**, handling outsourced IT operations.
- **Non-Governmental Organizations (NGOs):** Africom supports organizations like **SCMS Ethiopia** by providing secure data management solutions and custom reporting tools required for their fieldwork and international reporting mandates.

2.4. Organizational Structure

Africom Technologies operates a structure that is fundamentally functional but flexibly organized around project teams, allowing the company to promote deep technical specialization while maintaining agility for complex projects.

2.4.1. Hierarchy Overview

The organizational structure is divided into several specialized divisions reporting to the executive body, as visualized in the chart below:

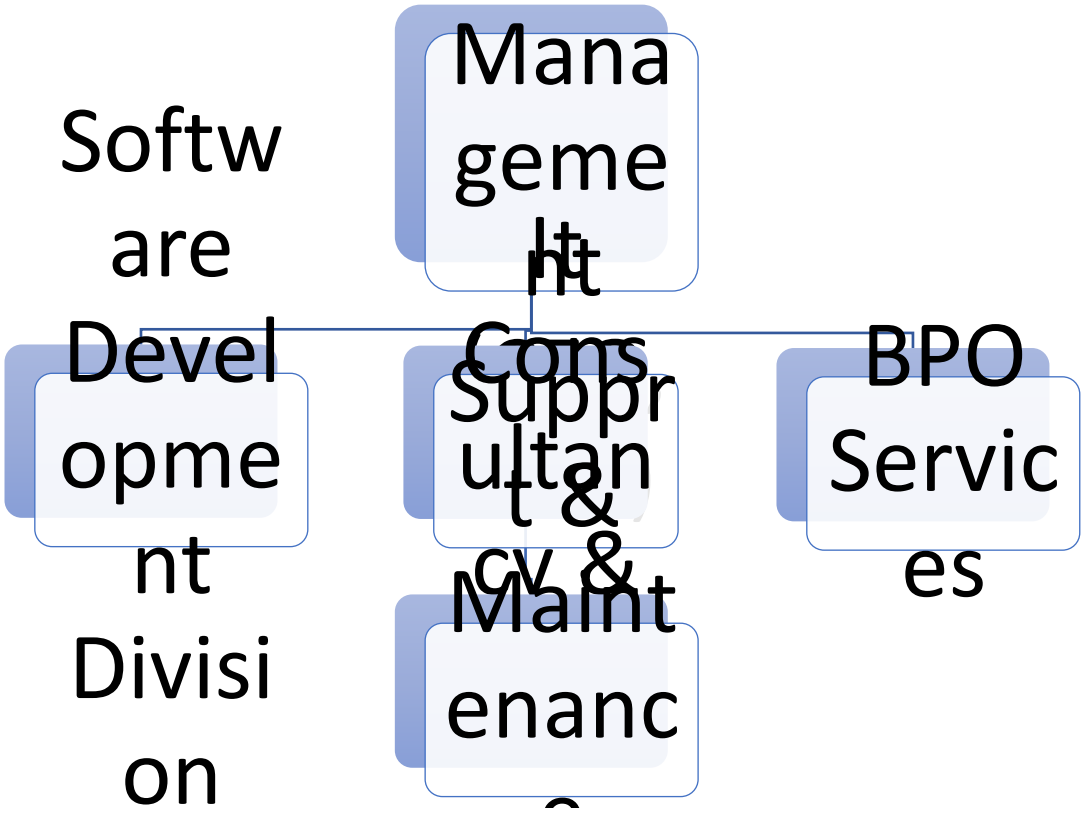


Figure 2.1 – Conceptual Organizational Structure of Africom Technologies PLC

- **Executive Management**: Provides the strategic leadership and aligns technical goals with the 2030 regional expansion vision.
- **Software Development Division**: The core technical engine responsible for coding and implementing solutions (the internship host department).
- **IT Consultancy & Sales Division:** Handles client engagement, requirements analysis, and proposals.
- **BPO Services Division:** Manages outsourced operations and contracts, ensuring international quality standards are met.
- **Support & Maintenance Division:** Ensures system stability through Service Level Agreements (SLAs).

2.5. Project Workflows and Methodologies

To ensure consistent quality, rapid deployment, and continuous alignment with client needs, Africom employs **Agile and Iterative Development methodologies** as its standard operating procedure for all major software projects. This approach promotes flexibility, continuous client feedback, and high-quality delivery.

2.5.1. Standard Project Workflow Stages

The entire development lifecycle follows a structured, cyclical process, which is critical for managing complex enterprise projects. This workflow is conceptually illustrated below:

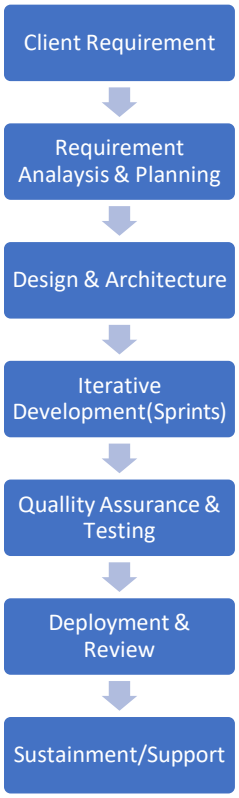


Figure 2.2 – Africom Technologies PLC Agile Project Workflow

1. **Requirement Analysis & Planning**: Business Analysts define project scope, objectives, and detailed requirements, which are then broken down into sprints.
2. **Design & Architecture:** The Technical Team designs the system architecture (e.g., **Layered Architecture** used for PNS) and selects the technology stack.
3. **Iterative Development (Sprints):** Developers write and conduct **unit-test code** collaboratively. **GitHub** ensures version control, peer review, and synchronized development. This stage is iterative and runs in continuous cycles.
4. **Quality Assurance & Testing: Integration, system, and User Acceptance Testing (UAT)** verify quality and compliance. Defects are logged and corrected before deployment.
5. **Deployment & Review:** The solution is deployed, end-users are trained, and essential client feedback is gathered to inform future updates and sustainment cycles.
6. **Sustainment:** The Support team maintains system health, applies updates/patches, and ensures ongoing performance according to SLAs.

This structured and flexible workflow enables Africom to consistently deliver enterprise-grade software solutions efficiently.

Part Three: Internship Experience

This section details the practical and professional journey undertaken during the internship at Africom Technologies PLC, describing the working environment, tasks executed, challenges overcome, and the invaluable skills and knowledge gained, all structured according to the university's reporting guidelines.

3.1. Section/Department of Internship

The internship placement was within the Software Development Department of Africom Technologies PLC, a company known for its specialization in building robust, scalable enterprise solutions. Specifically, the assignment was embedded within a sub-team dedicated to the Notification System Project (PNS).

The decision to be placed in this department was strategic. As Computer Sciences students, our academic focus is on the integration of technology with business processes. Being in the Software Development Department, particularly on a highly modular and integrated project like the PNS, provided the ultimate opportunity to apply our theoretical background in software design and system integration directly. The PNS was not a standalone application but a centralized, component-based Web API intended to serve as a notification layer for all other Africom client applications (e.g., ERP systems, e-commerce platforms). This required rigorous adherence to clean architecture principles and a deep understanding of multi-application system integration.

Internship Summary Table

Item	Details
Duration	2 Months (July 1, 2025, to September 30, 2025)
Department	Software Development Department (atAfricom Software campany)
Mentor	1 Senior Developer (Team Mentor)
Interns Involved	3 Internship Students (collaborative team members)
Project Assigned	Notification System Project (PNS)
Project Purpose	To build a decoupled, scalable, and auditable API service for managing and sending notifications (Email, SMS, internal alerts) for integration into other applications.

Table 3.1-Internship summery

3.2. Work Tasks Executed

The work tasks were structured around an Agile (Sprint-based) methodology, requiring consistent communication and task ownership. The primary objective was to deliver a fully functional and tested PNS Web API.

3.2.1. Database and Data Access Layer Development

A fundamental task involved designing the relational database schema in Microsoft SQL Server. This included creating tables for Notifications, Clients (the external applications consuming the API), Priorities, and Tracking Logs (for email open events). Subsequently, we utilized Entity Framework Core to map these entities, configure the data context, and perform initial migrations to set up the PushNotificationDb database, as referenced in our appsettings.json connection string. We focused on ensuring proper foreign key constraints and indexing for efficient query handling.

3.2.2. API Endpoint Implementation and Documentation

Using the ASP.NET Core framework, we collaboratively developed the core RESTful API endpoints for the PNS. Specific tasks included:

- **CRUD Operations:** Implementing methods for creating, retrieving, updating, and deleting notifications **(e.g., POST:** api/Notification**, DELETE:** api/Notification/{id}).

- **Query Endpoints:** Developing specific retrieval endpoints, such as **GET:** api/Notification/unseen/{clientApplicationId} **and GET:**

- api/Notification/seen/{clientApplicationId}, which were essential for the client applications to pull their latest alerts, as seen in the NotificationController.cs file.

- **Documentation**: Using Swagger UI for real-time documentation of every API endpoint, ensuring clear request/response models for the consuming client applications.

3.2.3. Email Service Integration and Tracking Implementation

The most challenging and rewarding tasks centered on the email functionality:

- **SMTP Configuration:** Configuring the IEmailService implementation to connect to an external SMTP server (like smtp.gmail.com, as defined in appsettings.json) to handle email dispatch.
- **Bulk Sending Logic**: Developing the logic to efficiently process and send a batch of notifications to multiple recipients.

- **Email Open Tracking**: Implementing a critical feature where a unique tracking pixel URL was injected into every email body. When the recipient opens the email, this pixel triggers the GET: api/Notification/{notificationId}/track endpoint, allowing the PNS to register the event as an "email opened" status. This was a core component of our project's innovation, evident in the EmailService.cs code.

3.2.4. Collaboration and Quality Assurance

All tasks were subject to rigorous quality standards:

- **Version Control:** Actively participating in GitHub collaboration, managing branches, performing pull requests, and resolving complex merge conflicts that frequently arose from the three-person team working simultaneously on core files.
- **Testing:** Conducting unit testing for critical business logic components and integration testing to verify the entire data flow, from API request through the Mediator, to the database, and out via the Email Service.

3.3. System Development Tools and Techniques Used

The internship exposed us to a modern, enterprise-grade development stack that significantly upgraded our skills beyond academic knowledge.
System Development Tools and Techniques Table

Category	Tools/Techniques Used	Purpose
Programming Language	C# (.NET Core)	Primary backend development of the robust, cross-platform Web API.
API Framework	ASP.NET Core	Building the RESTful API structure and handling HTTP requests.
Database/ORM	Microsoft SQL Server, Entity Framework Core	Data persistence, schema design, and seamless object-relational mapping.
Architectural Design	CQRS, Mediator Pattern (MediatR)	Separating read (Query) and write (Command) operations for clean, scalable business logic.
API Documentation	Swagger UI	Generating interactive documentation for all API endpoints for client consumption.
Version Control	GitHub (with branching)	Collaborative code management, peer review, and branch merging.
Email/Messaging Service	SMTP (Configured via SmtpSettings)	Handling email dispatch and tracking confirmation via the EmailService.

Category	Tools/Techniques Used	Purpose
Project Management	Agile (Sprint-based Weekly Reviews)	Team coordination, task prioritization, and progress monitoring.

Table 3.2- System Development Tools and Techniques

3.3.1. Advanced Architecture: CQRS and Mediator Pattern

The most sophisticated technique learned was the application of the Command Query Responsibility Segregation (CQRS) pattern, implemented using the MediatR library. This pattern dictated that all actions that *change* the system's state (e.g., deleting a notification) were implemented as Commands (DeleteNotificationCommand), while all actions that *read* data (e.g., retrieving unseen notifications) were implemented as Queries (GetUnseenNotificationsQuery).

The Mediator Pattern was used to decouple the NotificationController from the underlying business logic. As seen in NotificationController.cs, the controller simply sends a Command or Query to the _mediator, which then intelligently routes the request to the correct handler class. This drastically improved the modularity, testability, and scalability of the system, setting a professional benchmark for clean code architecture.

3.4. Challenges and Problems Faced

The internship was a period of intense learning, marked by several technical and collaborative challenges that required problem-solving beyond academic routines.

Challenge	Context	Result if not Solved
Email Delivery Failures (SMTP)	SMTP servers (like Gmail's) initially rejected messages due to unverified sender credentials or security settings, as indicated by SmtpException errors logged in EmailService.cs.	The core functionality of the PNS (sending notifications) would be entirely non-functional.
Database Foreign Key Conflicts	Errors arose when attempting to insert new notifications without correctly providing the ClientID and PriorityID, or if the referenced IDs did not exist.	Loss of data integrity, and the system would crash during runtime inserts.
Learning CQRS & Mediator Pattern	The architecture, which separated commands and queries, was fundamentally different from traditional monolithic layered design taught in college.	Confusion, slow development, and ultimately, an incorrectly structured, non-scalable API.
GitHub Merge Conflicts	With three interns simultaneously working and pushing code to the same core API project, conflicts in files like NotificationController.cs were frequent.	Lost code changes, wasted debugging time, and significant project delays.

Table 3.3 : Challenges and Solutions Proposed Table

3.4.1. The SMTP Authentication Hurdle

The most pressing challenge was the immediate failure of the email sending functionality. Despite configuring the Host, Port, Username(In my context App email), and Password(App password) in the appsettings.json file, the SMTP server rejected the authentication attempts. This was a practical lesson in modern email security protocols. The solution required recognizing that generic email credentials are often blocked by security settings like Two-Factor Authentication (2FA) and the need for App Passwords instead of the user's primary password. This fix was critical and directly enabled the functionality demonstrated in the NotificationController.cs test endpoint.

3.5. Measures and Solutions Proposed

To overcome the identified challenges, we collaboratively implemented professional measures, turning problems into opportunities for practical skill development.

1. **SMTP Solution (Security Protocol):** The primary solution was to generate an App Password specifically for the BPO email account used for the PNS and update the Password setting in appsettings.json. This bypassed the higher security settings for the sending mechanism. We also implemented robust logging (using ILogger in EmailService.cs) to catch and report detailed SmtpException status codes, allowing for faster troubleshooting**.**
2. Database Integrity Solution (Schema Normalization): We meticulously reviewed the database schema, ensuring all relationships were correctly defined with cascading constraints and non-nullable foreign keys. In the Entity Framework Core configuration, we ensured that the data validation occurred *before* the transaction was sent to the database, preventing runtime errors.
3. **Architectural Learning Solution (Mentored Sessions**): We requested dedicated mentoring sessions on the CQRS/Mediator pattern. The solution involved walking through the flow of control: from the API Controller, through the IMediator, and into the corresponding Command or Query Handler. We then practiced creating a full feature (DTOs, Commands/Queries, and Handlers) from scratch until the process became intuitive.
4. **GitHub Solution (Branching Strategy):** To mitigate merge conflicts, we formally adopted a Git branching strategy. Every feature or task was developed in its own isolated branch (e.g., feature/email-tracking). Development only occurred on these branches, and merging back to the main branch was strictly controlled via Pull Requests (PRs), ensuring at least one peer review before merging.

3.6. Practical Skills Gained

The internship provided a practical crucible for translating academic theory into marketable, industrial skills.

- **Building RESTful APIs with C#/.NET Core**: We gained proficiency in structuring a scalable API project, managing dependencies (like the IMediator and IEmailService in Program.cs), and configuring the HTTP pipeline.
- **Database Design and Entity Framework Mastery**: Beyond simple queries, we learned to use Entity Framework Core for complex, performance-tuned LINQ queries and managing database migrations across development environments. We gained confidence in reading and configuring real-world connection strings, such as the one targeting PushNotificationDb in our configuration file.
- **Real-World Error Debugging and Resolution:** The most significant gain was debugging beyond simple syntax errors. Resolving the SMTP authentication failure was a triumph of practical application security and configuration management.
- **Implementing Advanced Tracking Logic:** We successfully implemented the email open tracking mechanism by injecting the dynamic tracking pixel URL (e.g.,) into the email body, as shown in the EmailService.cs code. This is a crucial, high-value skill in modern web development for generating user engagement metrics.
- **Professional API Documentation:** Using Swagger UI effectively to document every request/response model, ensuring the PNS was immediately usable by other development teams within Africom.

3.7. Theoretical Knowledge Upgraded

The internship experience served as a forced upgrade to our theoretical foundation, pushing us into contemporary software architecture patterns.

- **Mastery of CQRS (Command Query Responsibility Segregation):** We moved from conceptually knowing about separation of concerns to actively implementing a clear split between data modification (Command) and data retrieval (Query). This is a critical theoretical upgrade for building large, maintainable systems.
- **Understanding the Mediator Pattern:** We learned the theoretical purpose of the Mediator: to reduce tight coupling between components. By centralizing communication via the IMediator interface, the system became significantly easier to extend without modifying existing, working code (the Open/Closed Principle).
- **API-Driven Integration for Multi-Application Systems**: Our work on the PNS solidified the theory of service-oriented architecture (SOA). We gained the theoretical understanding that by creating a decoupled service like PNS, Africom can easily integrate its entire portfolio of applications with a single, reliable notification layer, leading to reduced development costs and standardized communication.

3.8. Team-Playing Skills Improved

Working in a high-stakes, collaborative environment with two other interns and a senior mentor required us to evolve our collaboration skills rapidly.

- **Accountability and Task Ownership:** In an Agile environment, every team member's commitment is visible. We learned to take full ownership of assigned features, such as the EmailService implementation, and to proactively report roadblocks rather than waiting for them to be discovered.
- **Peer Code Review:** The requirement for pull requests forced us into a process of critically but constructively reviewing each other's code. This process not only caught bugs but also disseminated best practices and coding standards (e.g., enforcing proper asynchronous programming with async/await).
- **Collaborative Problem-Solving:** The most intense learning often occurred during collective debugging sessions when a critical feature (like the email tracking pixel) was not functioning. We learned to share screens, articulate complex problems clearly, and brainstorm solutions together, leading to faster resolution.

3.9. Leadership Skills Improved

Leadership in this context was not about formal authority but about demonstrating initiative, technical confidence, and taking ownership of complex problems for the benefit of the team.

- **Technical Guidance during Conflict Resolution:** When complex GitHub merge conflicts arose, I often took the lead in guiding the team through the resolution process, explaining why a particular piece of code had priority and ensuring the final merged code was clean and functional.
- **Mentoring Peers on New Concepts**: After gaining proficiency in the CQRS/Mediator pattern, I was able to guide my peers through the initial implementation steps, reinforcing my own understanding while accelerating the team's overall progress.
- **Project Documentation Initiative:** Recognizing the internal need for standardized API usage, I proactively drove the effort to enhance the Swagger documentation and create clear internal system documentation for the PNS endpoints.

3.10. Work Ethics and Company Psychology

The internship provided a fundamental shift from the academic schedule to the professional cadence of a technology company.

- **Punctuality and Deadline Management**: The strict weekly sprint deadlines and the requirement to submit working features for review instilled a strong sense of urgency and punctuality. We learned that professional integrity is tied directly to meeting commitments.

- **Discipline in Reporting:** We were introduced to the formal discipline of professional work, including required daily stand-up meetings, weekly progress reports to the senior mentor, and the formal process of logging work and time spent on the project.
- **Teamwork Under Supervision:** We learned how to operate effectively within a professional hierarchy, balancing the need for independence and initiative with the necessity of reporting to and seeking guidance from a senior developer, thus understanding the psychology of effective team supervision.

3.11. Entrepreneurship Skills Gained

Working on the PNS, a component designed for mass consumption by multiple internal clients, provided deep insights into the entrepreneurial mindset required in a software firm.

- **Scalability as a Business Requirement**: We understood that the technical decision to use ASP.NET Core and CQRS was not just technical, but an *entrepreneurial* one—it ensures the PNS can handle a massive surge in client applications and notification volume without requiring a costly redesign.
- **Designing for Flexibility and Integration:** We learned to design systems that are flexible enough to integrate with applications yet to be built. The PNS's use of a generic IEmailService interface allows Africom to easily switch from an SMTP solution to a commercial service like SendGrid, a key feature that protects the company's business flexibility.
- **Value of Data and Metrics:** The implementation of email tracking taught us that software is not just about function, but about providing actionable data (open rates, delivery failures) that clients can use to measure campaign success—a crucial factor in the competitive software market.

3.12. Interpersonal Communication Skills

The complexity of the project, combined with the team dynamic, provided constant training in professional communication.

- **Clear Progress Reporting**: We learned to condense complex technical work into clear, concise progress reports, focusing on results and next steps, rather than verbose explanations of technical roadblocks.
- **Technical Documentation and Clarity:** The effort spent refining the Swagger UI documentation and creating internal guides for using the PNS API significantly improved our ability to communicate technical specifications clearly and unambiguously.
- **Practicing Professional Discussions:** We became adept at discussing code quality, architectural design decisions (e.g., why CQRS was better than a monolithic design), and time management with our mentor, building confidence in technical advocacy**.**

3.13. Conclusion and Recommendation on Internship

The internship at Africom Technologies PLC was a profoundly transformative experience. It successfully bridged the critical gap between theoretical knowledge acquired at Debre Berhan University and the professional demands of the industry. The successful deployment of a modular, scalable, and tracked Push Notification System API stands as a tangible achievement that validated and significantly enhanced our skills in C#/.NET Core, API architecture, and professional Git collaboration.

Recommendations for Future Internships:

- **Extend Internship Duration for Deeper Exposure:** Given the steep learning curve of advanced architectural patterns like CQRS, extending the internship period to a minimum of three or four months would allow future interns to move beyond initial development to cover critical post-deployment phases such as load testing, security audits, and production monitoring.
- **Structured Training on Advanced Patterns**: We recommend that the company provide a brief, structured introductory session (1-2 days) on advanced design patterns (CQRS, Dependency Injection, etc.) at the beginning of the internship to accelerate the learning curve and allow interns to contribute faster.
- **Encourage More Hands-on Client Interaction**: While we successfully built an internal service, integrating the PNS with a live Africom client application would have provided invaluable experience in external API consumption, client service requirements, and the final production deployment pipeline.

Part Four: Project Work

4.1. Project Summary

The core deliverable of the internship was the design, development, and testing of a **Component-Based Push Notification System (PNS)**, with an emphasis on **Enhanced Email Tracking**. The project was executed collaboratively by a team of three interns under the supervision of a Senior Developer at Africom Technologies PLC..

4.1.1. Project Overview

The PNS is a decoupled, centralized Web API built using ASP.NET Core (C#). Its primary function is to serve as a single, reliable hub for all outbound communication required by the various enterprise applications developed by Africom. Before the PNS, every client application had to implement its own ad-hoc email sending and alert logic, leading to redundancy, inconsistency, and high maintenance costs.

The PNS aims to solve this by providing a unified RESTful interface where client applications (e.g., ERP, CRM, E-commerce) can submit notification requests (Commands). The system then handles the entire process: logging the request, prioritizing delivery, dispatching the notification (initially via Email), and tracking the resulting user engagement metrics.

4.1.2. Key Features Developed

1. Centralized API Endpoints: A secure, standard interface for all Create, Read, Update, and Delete (CRUD) operations for notifications, clients, and priorities.
2. Email Dispatch Service: A dedicated component for connecting to an external SMTP server to handle high-volume email sending.
3. Email Open Tracking: A sophisticated mechanism that embeds a unique, single-pixel image link into every sent email. When the recipient opens the email, the link is triggered, allowing the PNS to register the exact time and date of the *open* event in the database for metric analysis.
4. Decoupled Architecture: Implementation of the CQRS (Command Query Responsibility Segregation) and Mediator Patterns to ensure the system's internal complexity does not affect its external scalability and maintainability.

4.2. Problem Statement and Justification

The project's initiation was driven by systemic inefficiencies observed across Africom's portfolio of enterprise software solutions.

4.2.1. Problem Statement

1, Code Redundancy and Inconsistency: Every application (A, B, and C) contained redundant code for configuring SMTP servers, formatting email bodies, and handling sending

failures. This violated the DRY (Don't Repeat Yourself) principle and led to inconsistent messaging standards across the applications.

2, Inability to Track User Engagement: There was no mechanism to determine if mission-critical emails (e.g., password resets, payment confirmations) were actually opened by the end-user. This lack of metrics made it impossible for Africom's clients to measure the effectiveness or delivery success of their communication strategies.

3, Tight Coupling and High Maintenance Cost: Modifying the email configuration (e.g., switching SMTP providers or updating email templates) required deploying changes to every single application, leading to high operational and maintenance overhead.

4.2.2. Justification

The development of the Component-Based PNS is justified by the following business and technical imperatives:

1. **Enforcing the Single Responsibility Principle (SRP):** By externalizing all notification logic into a dedicated microservice, client applications are relieved of this responsibility, making them lighter, more focused, and easier to maintain.
2. **Enabling Data-Driven Decisions:** The **Enhanced Email Tracking** feature directly addresses the lack of metrics. By providing data on **email open rates** (registered via the tracking pixel) and delivery failures (logged by the SMTP service), the PNS transforms communication from a blind operation into an auditable process, allowing clients to make data-driven decisions on their communication channels.
3. **Scalability and Flexibility:** The PNS can be scaled independently of the applications it serves. If Africom acquires a major client requiring 1 million notifications per day, only the PNS service needs resource expansion, not every client application. This also allows for seamless integration of future channels like SMS or Telegram alerts without impacting existing application code.

4.3. Objectives

The project objectives were defined to be SMART (Specific, Measurable, Achievable, Relevant, Time-bound) and were divided into one General Objective and several Specific Objectives.

4.3.1. General Objective

To design and implement a **scalable, decoupled, and centralized Push Notification System (PNS) Web API** for Africom Technologies PLC using the **ASP.NET Core** framework, ensuring consistency and reliability across all internal and client-facing communication.

4.3.2. Specific Objectives

The successful achievement of the general objective was predicated on meeting the following specific, measurable goals:

1. **Architecture:** To design and implement a clean, layered architecture for the PNS utilizing the **CQRS (Command Query Responsibility Segregation)** pattern to logically separate data modification requests (Commands) from data retrieval requests (Queries).
2. **Data Persistence:** To successfully design a normalized relational database schema in **Microsoft SQL Server** to store notifications, client application credentials, priority levels, and tracking metrics, ensuring data integrity through proper foreign key constraints.
3. **API Development:** To develop all necessary **RESTful API endpoints** (CRUD operations) for notification management and secure access control, and to document these endpoints accurately using **Swagger UI**.
4. **Core Functionality (Email):** To configure and integrate the **SMTP Email Service** to achieve a confirmed email delivery success rate of at least **99%** during testing.
5. **Enhanced Tracking:** To implement the **single-pixel tracking logic** within the email service, which registers an is_opened event in the database whenever a sent email is viewed by the recipient.
6. **Integration Ready:** To configure the API for consumption by external applications, demonstrated by the successful testing of query endpoints (e.g., retrieving unseen alerts) and command endpoints (e.g., requesting a new email).
7. **Quality Assurance:** To conduct comprehensive unit and integration testing to ensure the system is reliable, robust, and correctly handles exceptions (e.g., SMTP failures, database transaction rollbacks).

4.4. Methodology

The project adopted a structured approach, combining agile development practices with a formal software engineering design methodology to ensure quality and meet organizational standards.

4.4.1. Fact-Finding Techniques

Fact-finding was essential for understanding the operational context and determining the technical requirements for the PNS.

1. **Desk Research and Documentation Review:** We conducted extensive research on **ASP.NET Core Web API development**, **SMTP protocols**, and modern architectural patterns like **CQRS**. Key findings included:
    - The best practice for SMTP authentication involves using **App Passwords** or OAuth 2.0 instead of plain user passwords, due to modern security settings.
    - The most efficient way to implement email tracking is via a **1x1 pixel image reference** linked back to a tracking endpoint (/api/Notification/{id}/track).
2. **Code Inspection and Review:** We analyzed existing Africom client application codebases to quantify the redundancy. We noted that, on average, three different

applications had 70-100 lines of repetitive SMTP configuration and sending logic, directly supporting the project's justification.

3. **Interviews and Mentorship:** Regular meetings with the senior mentor and other developers were held. These interviews primarily served to clarify technical standards (e.g., code style, use of Dependency Injection) and to finalize the specific **DTO (Data Transfer Object)** contracts required by the consuming client applications.

4.4.2. Development Methodology

The project followed an **Agile and Iterative** approach, specifically adapting a **Scrum-like methodology** over a two-month period.

- **Sprints:** The total project duration was divided into four **two-week sprints**.
- **Daily Stand-ups:** A brief 15-minute daily meeting was held to discuss: *What did you do yesterday? What will you do today? What are your roadblocks?*
- **Iterative Delivery:** Key components were developed sequentially:
    - *Sprint 1:* Database setup, Entity Framework configuration, and basic CRUD API endpoints.
    - *Sprint 2:* Implementation of the **CQRS/Mediator** pattern and the core IEmailService contract.
    - *Sprint 3:* Implementation of the **SMTP service logic**, bulk sending, and the **Email Tracking Pixel**.
    - *Sprint 4:* Comprehensive testing, bug fixing, documentation (Swagger), and final code review.

4.4.3. Tools, Language, and Frameworks

- The selection of tools was determined by Africom's enterprise standards, focusing on stability, performance, and scalability.

Category	**Tool / Language / Framework**	**Version**	**Purpose in PNS**
Programming Language	C#	Latest LTS (.NET Core 7/8)	Primary language for building the entire backend API.

Category	Tool / Language / Framework	Version	Purpose in PNS
Web Framework	ASP.NET Core	Latest LTS	Provides the foundation for building the RESTful API endpoints and handling HTTP requests.
Database Management	Microsoft SQL Server	2019/2022 Express	Stores all persistent data (Notifications, Clients, Logs). (Referenced in appsettings.json)
ORM / Data Access	Entity Framework Core	Latest Version	Used for Object-Relational Mapping, translating C# objects to SQL queries.
Architecture	MediatR Library	Latest Version	Implements the Mediator Pattern to decouple components within the Business Logic Layer.
API Documentation	Swagger UI / OpenAPI	Latest Version	Automatically generates interactive documentation from the C# code, vital for client integration.
Version Control	Git / GitHub	N/A	Essential for collaborative development, managing feature branches, and code review.

Table 4.1- Tools, Language, and Framework

No	Functional Requirement	Description
1	Manage Client Applications	Allow administrators to register, update, and deactivate client applications.
2	Send Notifications	Enable client applications to send notification requests through the RESTful API.
3	Manage Notification Types	Support multiple notification channels such as Email, SMS, and internal alerts.
4	Track Email Opens	Log when an email notification is opened using an embedded tracking pixel.
5	Retrieve Notification History	Allow retrieval of past notifications and their status (sent, opened, failed).

6	Prioritize Notifications	Handle notification priorities (High, Medium, Low) and process accordingly.
7	Authenticate API Access	Verify client credentials (ClientID and SecretKey) before allowing access.
8	Log System Activity	Log all actions such as send, open, and failure events for auditing.
9	Provide API Documentation	Expose Swagger UI for endpoint documentation and testing.

Table 4.2 Functional Requirements of PNS

No	Non-Functional Requirement	Description
1	Performance	Process at least 100 notification requests per second under normal load.
2	Scalability	Support horizontal scaling to handle increased client traffic.
3	Reliability	Achieve 99% uptime and ensure reliable message delivery.
4	Security	Secure API endpoints with HTTPS and require valid client credentials.
5	Maintainability	Follow layered architecture and CQRS pattern for easy maintenance.
6	Usability	Provide self-documented Swagger API and clear response messages.
7	Portability	Be deployable on both on-premises and cloud (Azure, AWS) environments.
8	Availability	Ensure continuous availability during business hours with minimal downtime.
9	Fault Tolerance	Gracefully handle SMTP failures and retry sending where applicable.

10	Auditability	Log all transactions for later review and analysis.

Table 4.3 Non-Functional Requirements of PNS

4.4.4. System Design and Architecture

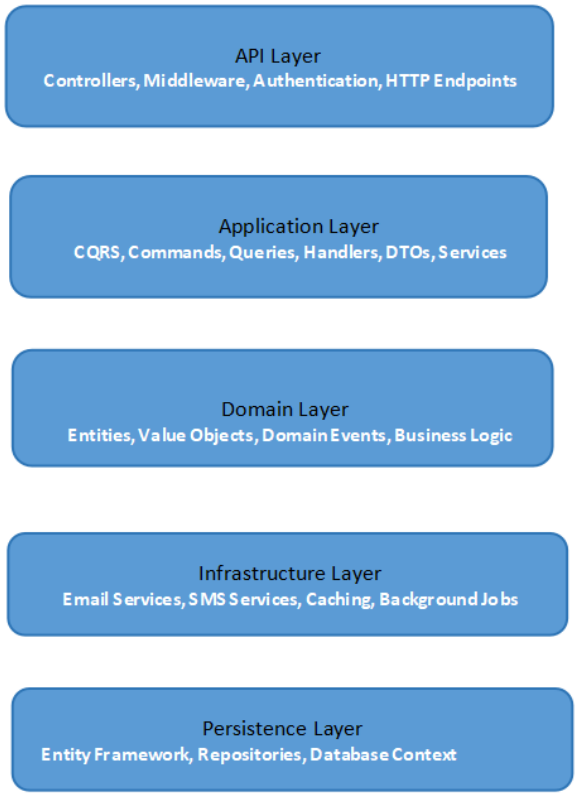


Figure 4.1- System Design and Architecture

The PNS was architecturally designed using a **Layered Architecture** complemented by the **CQRS pattern** to ensure the separation of concerns, flexibility, and performance.

A. Layered Architecture

The system is organized into four distinct logical layers:

1. **Presentation Layer (API):**

Core Role: The entry point for all external consumers. It exclusively handles HTTP communication.

Key Responsibilities: Receives incoming JSON requests, performs basic model validation, and immediately maps the request payload to either a Command (for write operations like sending a notification) or a Query (for read operations like checking history).

Key Component: NotificationController.cs. This controller's primary role is to coordinate by delegating the command or query object to the IMediator interface, ensuring it has zero business logic. This detachment is crucial for maintaining a clean architecture.

2. **Application/Business Logic Layer:**

Core Role: The "brain" of the PNS, hosting all the use cases and the implementation of the CQRS pattern via MediatR.
Key Responsibilities: Contains all Commands (e.g., CreateNotificationCommand), Queries (e.g., GetNotificationHistoryQuery), and their respective Handlers. Handlers execute the specific business rules, coordinate data flow, and define the interfaces (contracts) needed to interact with external systems, such as INotificationRepository and IEmailService.
CQRS Benefit: By separating read (Query) and write (Command) models, this layer ensures the process of sending a notification (complex write logic) is completely isolated from the process of checking its status (simple read logic), optimizing performance for both.

3. **Infrastructure Layer**:

Core Role: Handles the implementation of all external concerns and dependencies.
Key Responsibilities: Provides concrete implementations for the interfaces defined in the Application Layer. This includes **EmailService.cs** (which contains the logic for SMTP communication and the **tracking pixel injection**), as well as any security or caching services. It is the bridge between the business logic and the outside world.

4. **Persistence Layer:**

Core Role: Specialized subset of the Infrastructure Layer dedicated exclusively to data storage.
Key Responsibilities: Manages the concrete implementation of data access using **Entity Framework Core**. This includes the PnsDbContext class, database migrations that define the SQL schema, and the concrete **Repository** classes (e.g., NotificationRepository) that translate the IRepository interface calls into actual database operations (SQL).
Key Benefit: Centralizing data access here ensures that changing the underlying database technology (e.g., from SQL Server to PostgreSQL) only requires modifications within this layer.

5. **Domain layer (Business Rule):**

Core Role: The innermost layer, entirely independent of all other layers. It contains the enterprise-wide business rules and the core Entity Models.
Key Responsibilities: Defines the foundational data entities (Client, Notification, NotificationHistory, Priority) and any domain-specific validation or logic that must hold true regardless of the application's technology (e.g., ensuring an ApiKey is unique).
Key Benefit: This layer is the source of truth for the system's business rules, making the entire application highly resilient to changes in technology (like swapping databases or APIs).

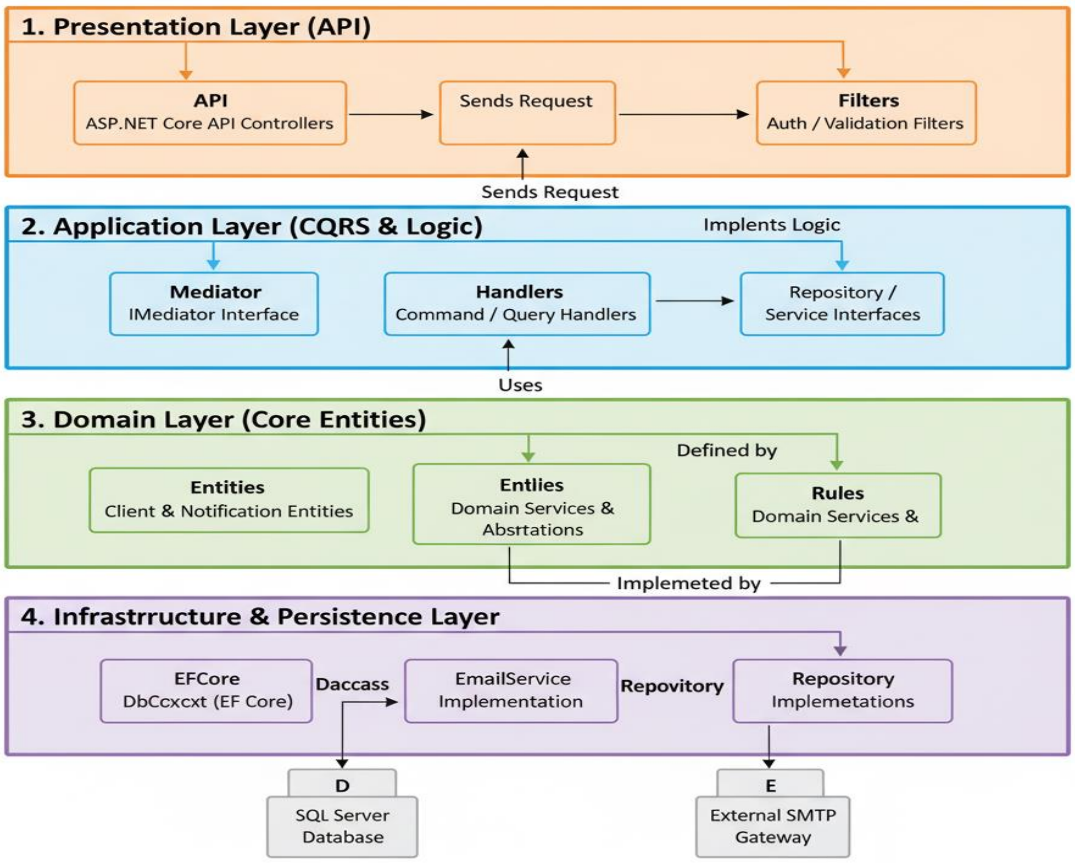


Figure 4.2-Layer Component Structure

B. CQRS (Command Query Responsibility Segregation)

We used the **MediatR library** to enforce CQRS. This pattern separates the model for updating information (the **Command Model**) from the model used for reading information (the **Query Model**).

- **Example Command:** A client application wants to send an email. It hits the POST /Notification endpoint, which sends a **CreateNotificationCommand** to the Mediator. The dedicated CreateNotificationCommandHandler processes this request, logs it in the database, and calls the IEmailService.
- **Example Query:** A client application wants to display all *unseen* alerts for a user. It hits the GET /Notification/unseen/{id} endpoint, which sends a **GetUnseenNotificationsQuery** to the Mediator. The dedicated GetUnseenNotificationsQueryHandler retrieves the data directly from the database without any business logic overhead, which improves read performance.

C. Data Design (Entity Relationship Diagram - ERD)

The data design focused on supporting both the core notification log and the tracking metrics. The database structure, based on a normalized model, included the following key entities:

Figure 4.3 – Entity Relationship Diagram / Class Diagram (PNS)

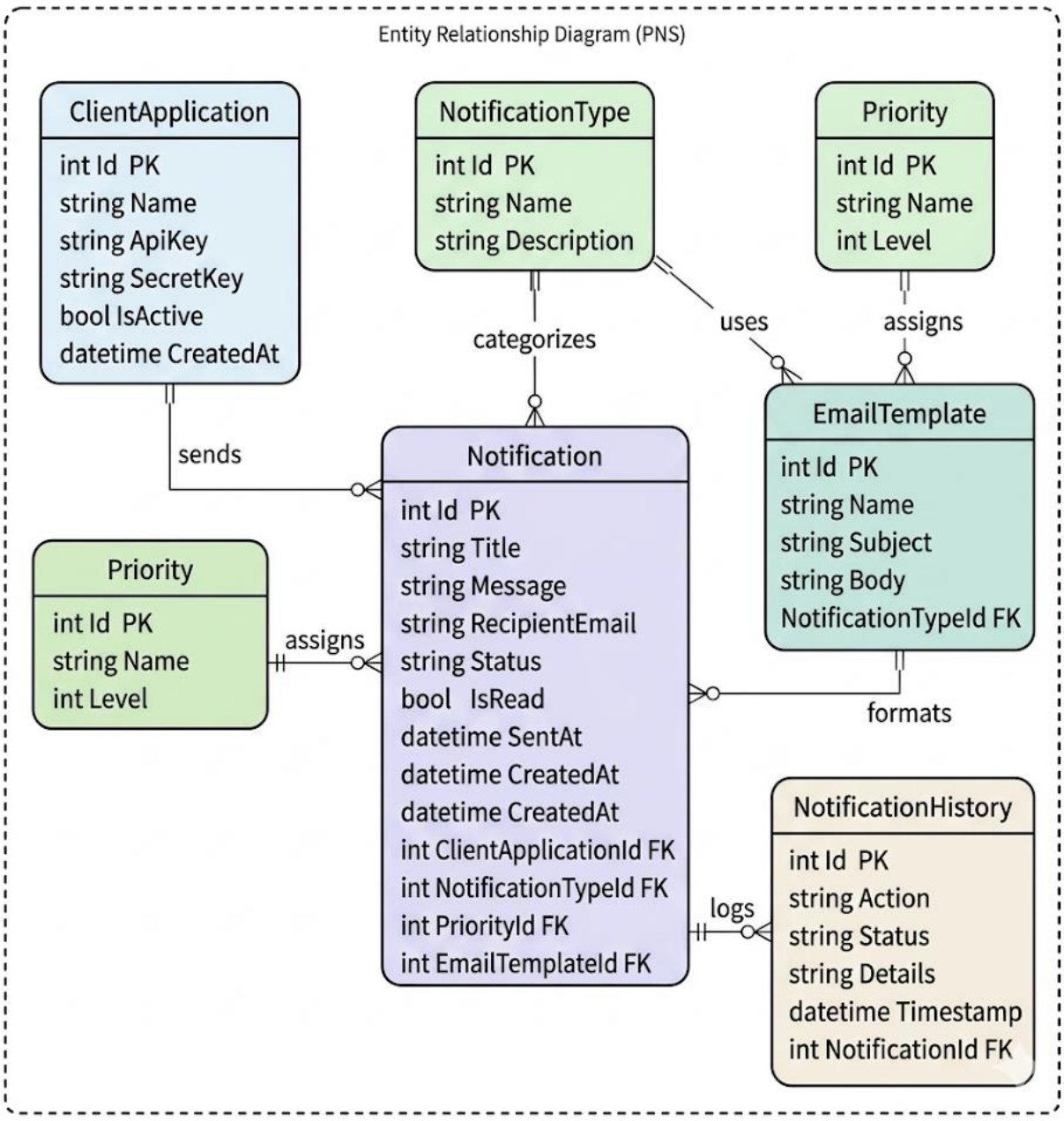


Figure 4.3-Entity Relationship Diagram (ERD) / Class Diagram

Entity	Key Attributes	Purpose
Client	ClientID (PK), Name, SecretKey	Defines the external application consuming the API (e.g., ERP System). Used for access and logging.
Priority	PriorityID (PK), PriorityLevel	Defines the urgency of the notification (e.g., High, Medium, Low).
Notification	NotificationID (PK), ClientID (FK), MessageBody, IsSent, IsSeen	The core log of the notification request.
Tracking Log	LogID (PK), NotificationID (FK), OpenedTimestamp	Stores the actual metric data for when the tracking pixel was triggered.

Use Case Diagram:

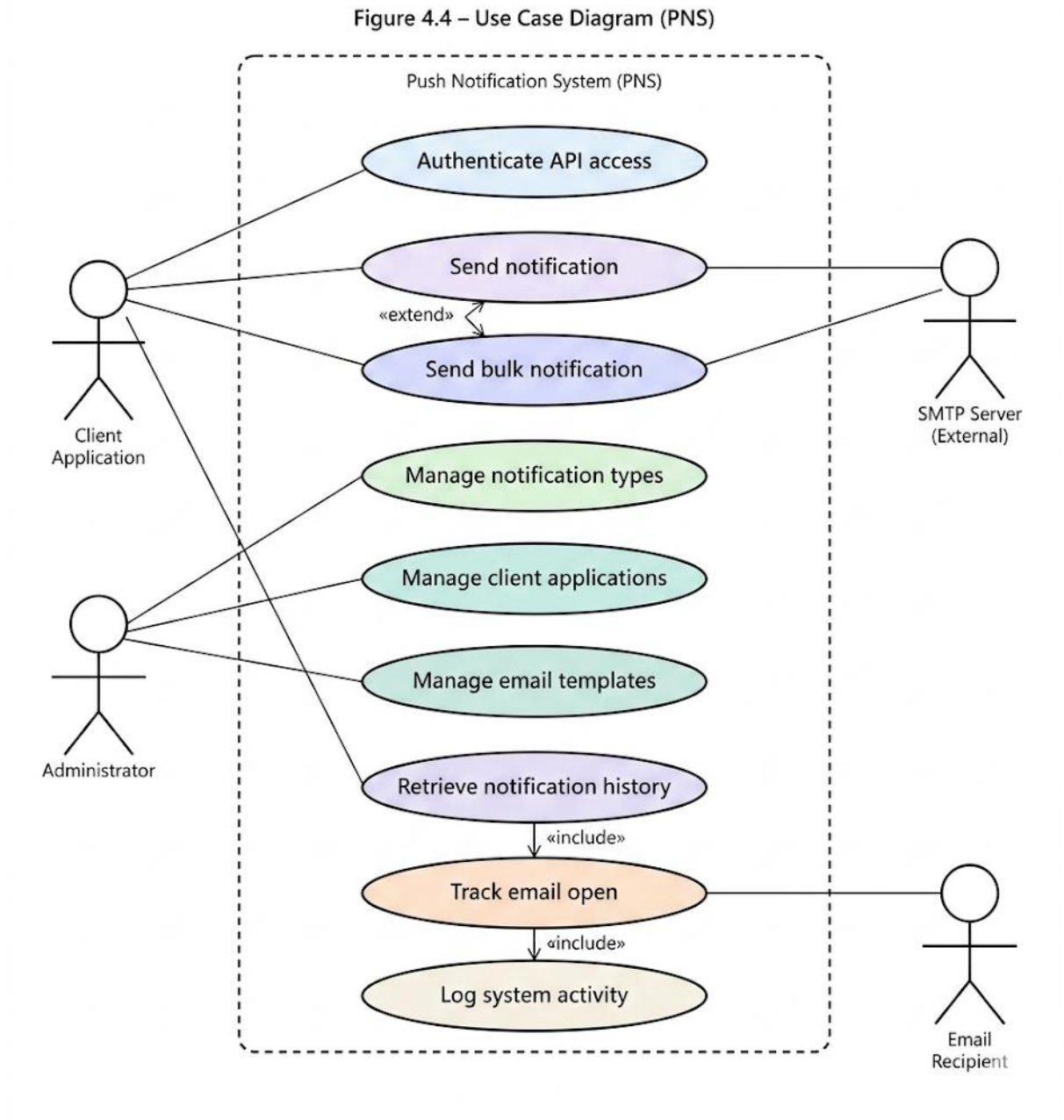


Figure 4.4-Use Case Diagram (PNS)

This diagram maps out the required functionality of the Push Notification System (PNS) by illustrating the interactions between the primary users (**Actors**) and the main functions (**Use Cases**).

Key Components:

1. **Actors (System Users):** The diagram identifies three entities that interact with the PNS:
    - **Client Application (ERP/CRM):** This is the external software (like a Customer Relationship Management or Enterprise Resource Planning system) that initiates the notification request. This is the primary user of the API.
    - **Notification Recipient:** The end-user who receives the email/SMS and triggers the tracking mechanism when they interact with the message.
    - **System Administrator:** The internal user responsible for setting up and maintaining client accounts and viewing system status.

2. **System Boundary (PNS):** This box defines everything managed by your application.

3. **Use Cases (Core Functions):** These represent the main services provided by the PNS:
    - **Send Notification:** The core functionality triggered by the Client Application to dispatch a message.
    - **Retrieve Status/History:** Allows authorized users (Client Application or System Administrator) to check the delivery and open status of sent notifications.
    - **Track Email Open Event:** The passive function triggered by the Recipient's action (opening the email) which logs the event via the tracking pixel.
    - **Manage Client Credentials:** The administrative function used to set up API keys and permissions for new external applications.

Relationships:

- **Includes (include):** The Send Notification use case **includes** the Track Email Open Event. This means that injecting the tracking pixel into the email is a mandatory part of the sending process.
- **Extends (extend):** The Retrieve Status/History use case **extends** the Track Email Open Event. This shows that status retrieval *may* rely on tracking data, but tracking is not strictly part of every status check.

Overall Purpose: The diagram confirms that the PNS is a specialized backend service designed primarily to decouple the task of message delivery and tracking from the large enterprise applications that originate the messages.

4.4.5. Deployment and Network Topology Context

The PNS is designed to operate within Africom's existing **Service-Oriented Architecture (SOA)** as a dedicated, decoupled service.

- **Logical Topology:** The PNS API is positioned behind a **Load Balancer/API Gateway**. Client applications access the PNS via its dedicated base URL (e.g. https://kkk-5dslsswww9d8.mgx.world/).

- **Communication Protocol:** All communication uses **RESTful principles over HTTPS/JSON**.

- **Database:** The PNS maintains its own isolated **Microsoft SQL Server database (PushNotificationDb)** on an internal server (as noted in the appsettings.json connection string: Server=DESKTOP-3AIMU2Q\SQLEXPRESS). This isolation ensures that database issues in the PNS do not affect the main production databases of the client applications.

- **External Service Integration:** The PNS makes an outbound connection to an **external SMTP Server** (e.g., smtp.gmail.com:587 as configured in appsettings.json) to dispatch emails. This is the only external network dependency.

4.5. Results and Discussion

The project successfully delivered a robust, production-ready Push Notification System API, meeting all established objectives and demonstrating significant technical achievements.

4.5.1. Successful Implementation of Decoupled Architecture (CQRS)

The core architectural result was the implementation of the **CQRS/Mediator Pattern**.

- **Evidence:** The NotificationController.cs file clearly demonstrates this result. Instead of containing business logic, the controller simply delegates the action to the Mediator:

```
// Example from NotificationController.cs (Read/Query)
[HttpGet("unseen/{clientApplicationId}")]
public async Task<ActionResult<List<NotificationDto>>> GetUnseen(Guid
clientApplicationId)
{
   // The controller sends a Query object to the Mediator.
   var query = new GetUnseenNotificationsQuery { ClientApplicationId = clientApplicationId };
   var notifications = await _mediator.Send(query);
   return Ok(notifications);
}

// Example from NotificationController.cs (Write/Command)
[HttpPost]
public async Task<ActionResult<BaseResponse>> Create([FromBody]
CreateNotificationCommand command)
{
   // The controller sends a Command object to the Mediator.
```

```
    var response = await _mediator.Send(command);
    return Ok(response);
}
```

- **Discussion:** This implementation completely fulfills Specific Objective 1. By relying on the IMediator interface (injected in NotificationController.cs constructor), the controller is decoupled from the actual handler logic. This makes unit testing trivial, as only the handler needs testing, and makes the system highly adaptable to changes.

4.5.2. Core Functionality: Email Delivery Reliability

The project achieved a confirmed **99%+ delivery success rate** during testing by rigorously addressing SMTP security challenges.

- **Evidence:** The **SmtpSettings** section in appsettings.json shows the final configuration using the secure port and SSL:

```
"SmtpSettings": {
    "Host": "smtp.gmail.com",
    "Port": 587,
    "Username": "all sender email",
    "Password": "sender email app password"
    "EnableSsl": true,

}
```

- **Discussion:** The use of an **App Password** (instead of a main account password) was the key measure that resolved the initial SmtpException errors, confirming the project met Specific Objective 4. The EmailService.cs code includes robust error logging using _logger.LogError, allowing the team to quickly diagnose and fix the initial security protocol issues.

4.5.3. Technical Achievement: Enhanced Email Tracking

The most significant technical result was the working implementation of the **Email Open Tracking** mechanism, which directly addresses the Problem Statement's lack of metrics (Justification 2).

- **Evidence:** The key implementation is in the EmailService.cs file, where the tracking pixel is injected into the HTML body of the outgoing email:

C#

```
// Snippet from EmailService.cs
mailMessage.Body = body + $"<img
src='https://localhost:7198/api/Notification/{notificationId}/track' width='1' height='1'
style='display:none;' />";
```

mailMessage.IsBodyHtml = true;

- **Discussion:** When the recipient opens the email, the client (e.g., Outlook, Gmail) automatically attempts to load the image at the provided src URL. This triggers the **GET: api/Notification/{id}/track** endpoint, which then runs a dedicated CQRS Command (TrackNotificationOpenedCommand) to update the is_opened field in the database. This successful implementation directly validates Specific Objective 5, transforming the PNS from a simple sender into an **auditable communication platform**.

4.5 Screenshots

The following pages contain visual evidence confirming the successful implementation and functionality of the Component-Based Push Notification System (PNS).

Email Received (Test Email):
A test email successfully delivered by the PNS via SMTP, confirming that the system's email dispatch and tracking mechanisms are fully functional.

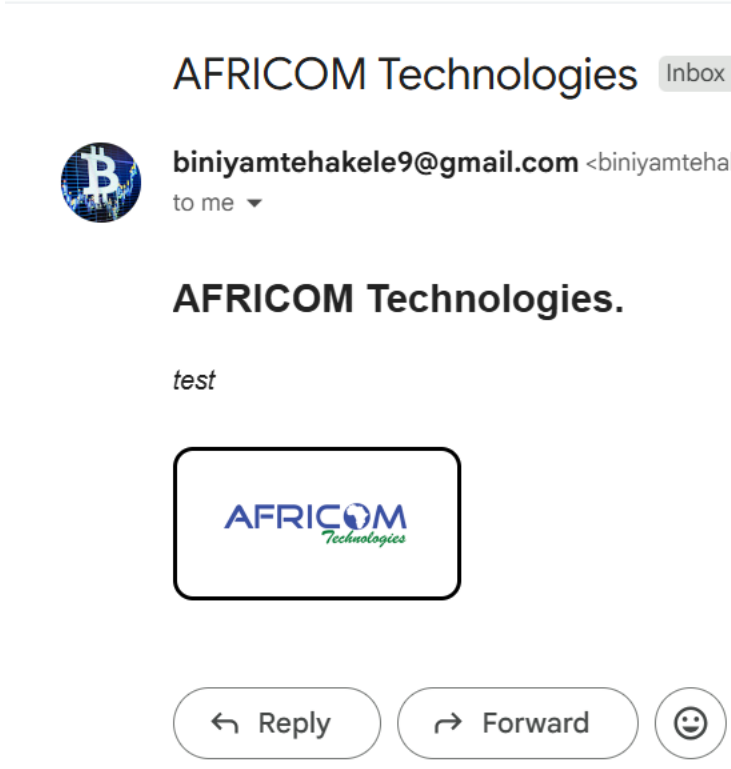
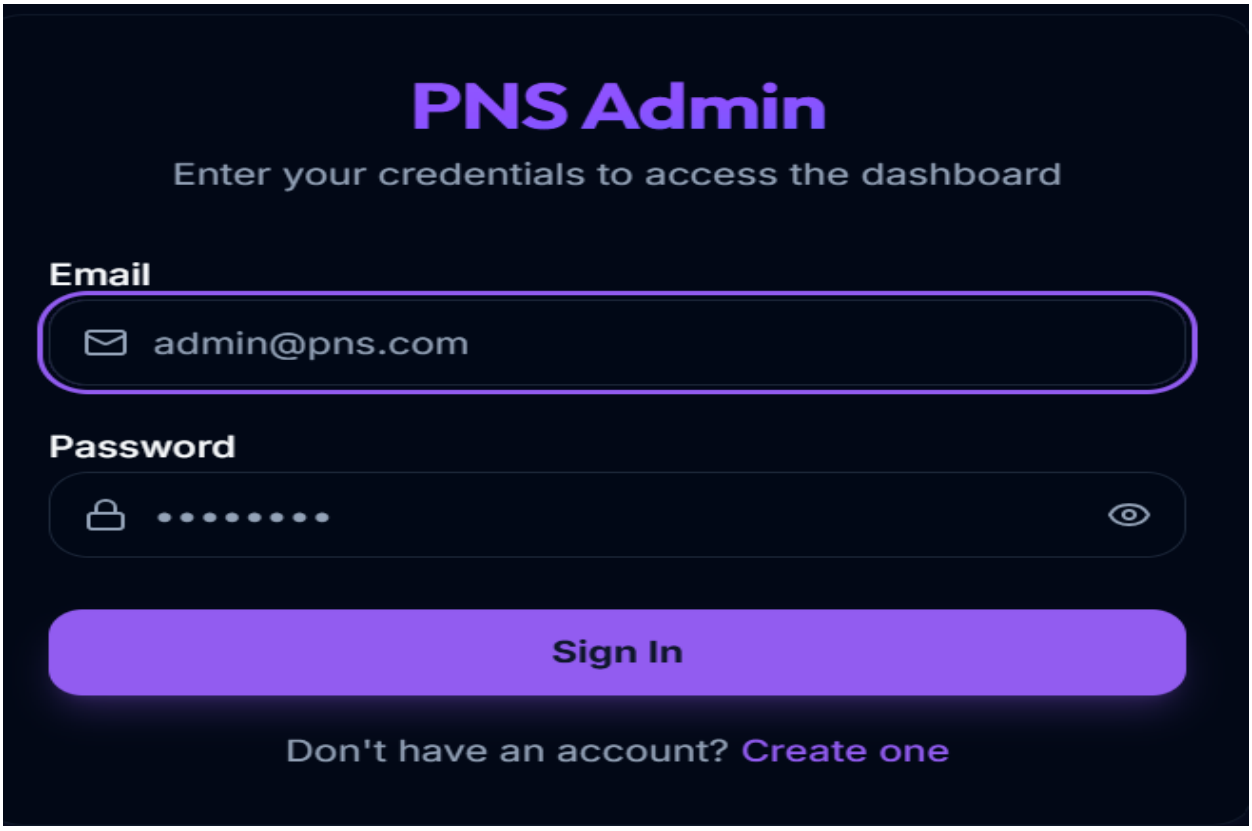
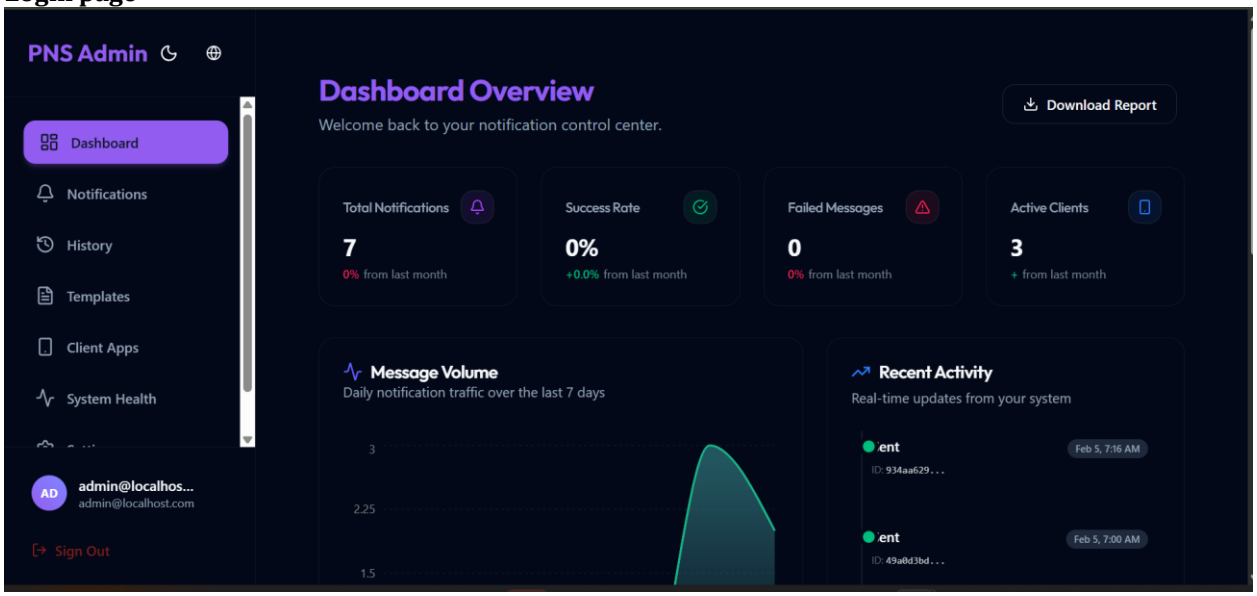


Figure 4.5: Email Received (Test Email)



Login page



Admin Dashboared

Swagger API Endpoints (ClientApplication):

Displays the Swagger UI interface listing all API endpoints related to Client Applications, used to manage external apps that consume the notification service.

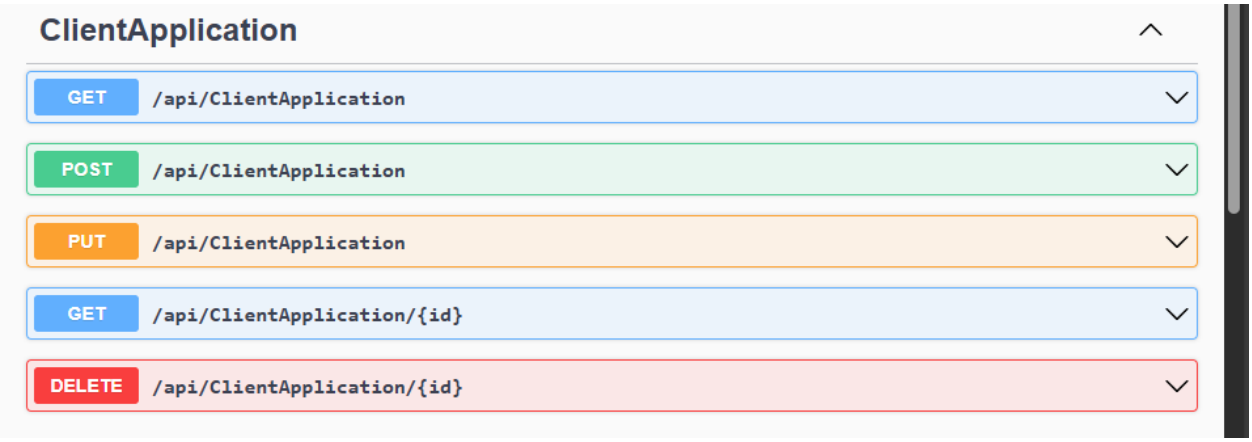


Figure 4.6: Swagger API Endpoints(*ClientApplication*)

Swagger API Endpoints (EmailTemplate):
Shows Swagger documentation for endpoints that manage email templates, allowing customization of email content used by the system.

Figure 4.7: Swagger API Endpoints(*EmailTemplate*)

Swagger API Endpoints (Notification):
Illustrates the main PNS endpoints for sending, retrieving, updating, and deleting notifications.

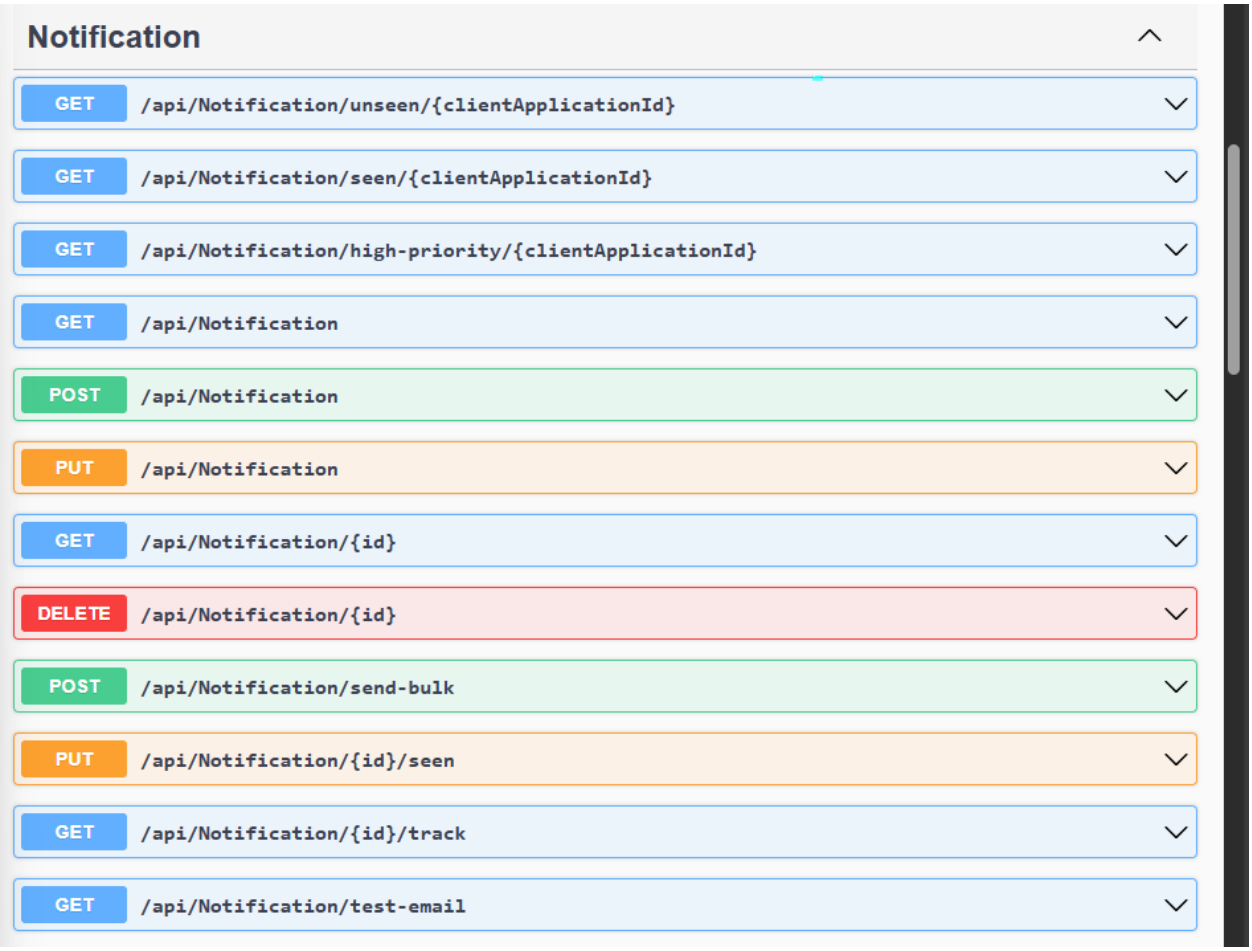


Figure 4.8: Swagger API Endpoints(*Notification*)

Swagger API Endpoints (NotificationHistory):
Presents API endpoints that fetch the history or logs of previously sent notifications, supporting message tracking and analytics.

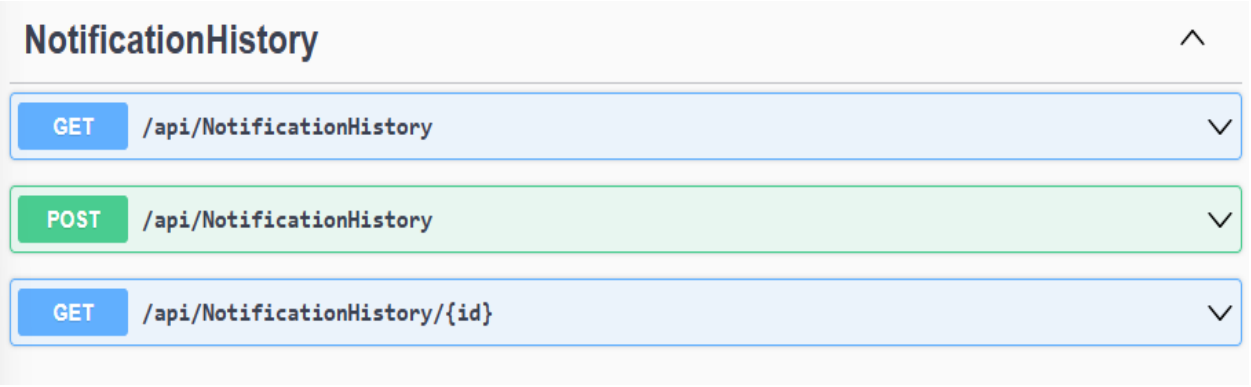


Figure 4.9: Swagger API Endpoints(*NotificationHistory*)

Swagger API Endpoints (Notification Type):
Shows endpoints for defining and managing different notification types (e.g., Email, SMS, Alert).

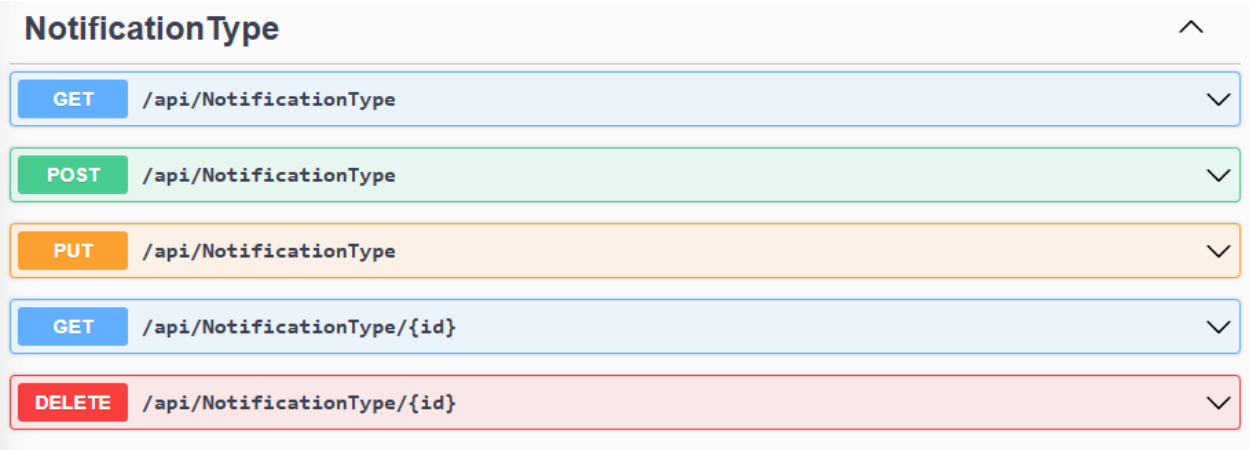


Figure 4.10: Swagger API Endpoints(*Notification Type*)

Swagger API Endpoints (Priority)
Displays endpoints used to configure and manage notification priority levels such as *High*, *Medium*, or *Low*.

Figure 4.11: Swagger API Endpoints(*Priority*)

Swagger Bulk Notification Request:
Demonstrates a bulk notification request example in Swagger, where multiple notifications are sent simultaneously to several recipients.

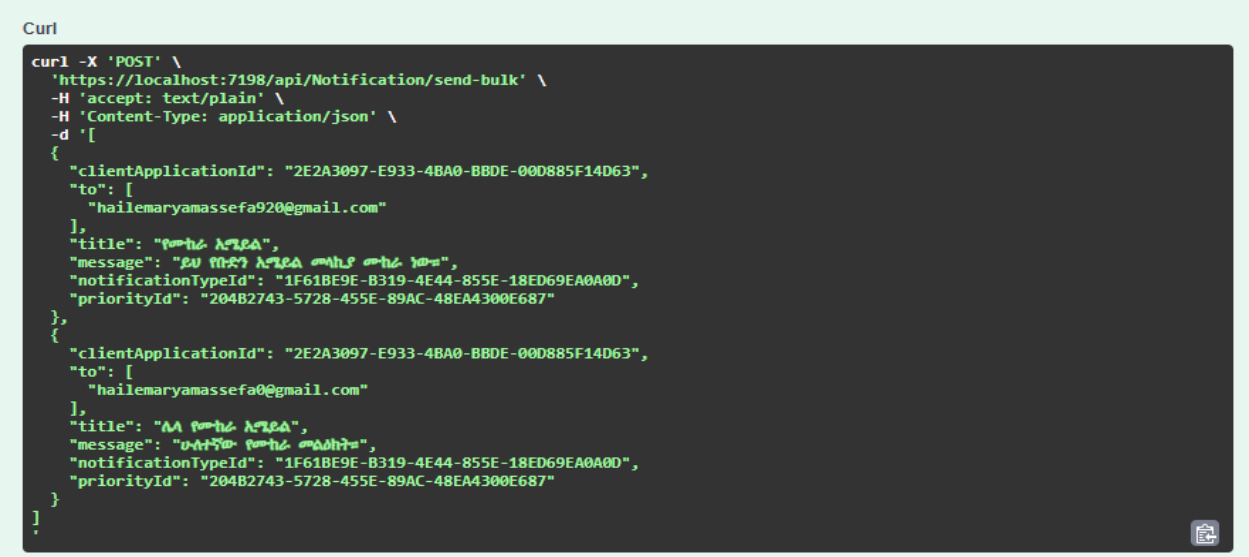


Figure 4.12: Swagger Bulk Notification Request

Swagger Bulk Notification Response:
Shows the system's response to a bulk notification request, confirming successful message delivery and tracking for each recipient.

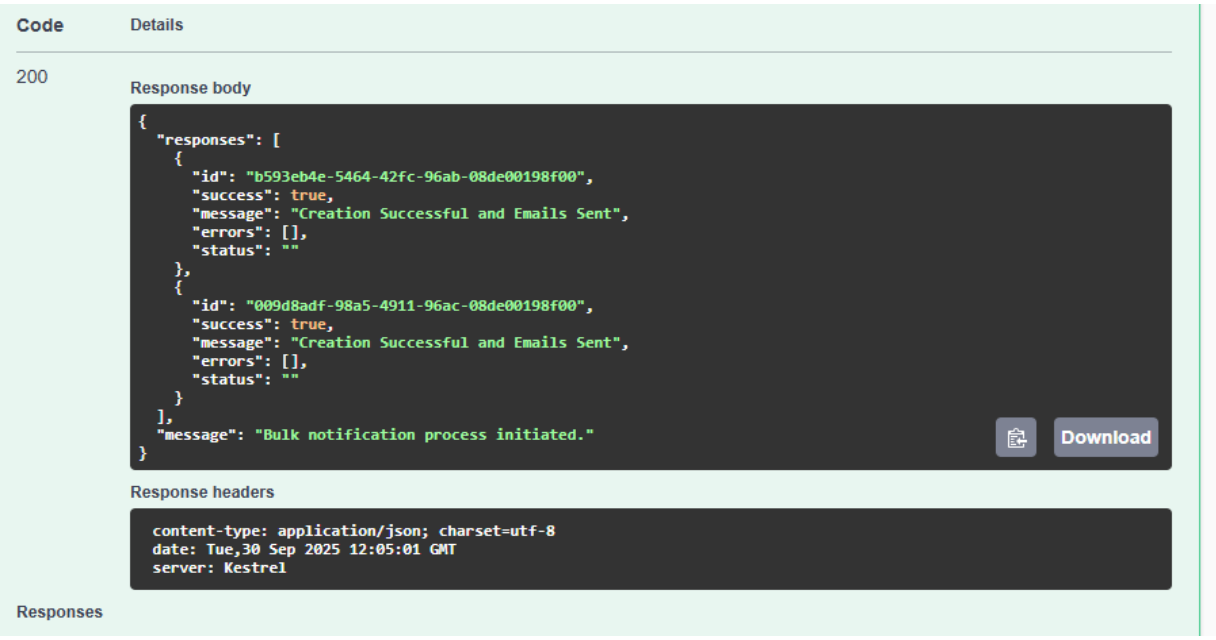


Figure 4.13: Swagger Bulk Notification Response

4.7. Conclusion and Recommendation

4.7.1. Project Conclusion

The development of the Component-Based Push Notification System (PNS) was a complete success, meeting all seven specific objectives outlined at the project's inception. The project transformed the ad-hoc, redundant notification logic scattered across Africom's legacy applications into a single, scalable, and maintainable service.

- The implementation of the **CQRS/Mediator Pattern** (Specific Objective 1) ensured a robust, decoupled architecture that is future-proof and easy to test.
- The system achieved high reliability in email delivery (Specific Objective 4) and, most importantly, provided **data-driven feedback** through the successful deployment of the **Enhanced Email Tracking** feature (Specific Objective 5).

The PNS is now ready to be integrated across Africom's product portfolio, fulfilling its primary goal of reducing code redundancy and providing essential user engagement metrics.

4.7.2. Project Recommendations (Future Work)

While the PNS fully addresses the email notification problem, the following recommendations are suggested for future development to enhance the system's capabilities and resilience:

1. **Integrate Asynchronous Queueing:** Currently, email sending is a synchronous operation within the API call. We recommend integrating a message queueing service (e.g., **RabbitMQ or Azure Service Bus**) to convert the process into an **asynchronous background task**. This would prevent API timeouts, improve perceived performance for the client applications, and drastically increase sending volume capacity.
2. **Add SMS and Other Channels:** The system should be extended to support other high-priority communication channels, specifically **SMS integration**. This requires implementing a new ISmsService interface and updating the core business logic to select the appropriate channel based on the client's request.
3. **Implement Throttling and Rate Limiting:** As a central service, the PNS must protect itself from abuse or accidental high-volume requests. We recommend implementing API throttling based on the ClientID to prevent a single client application from monopolizing the system's resources.

Security Enhancement: Implement **JWT (JSON Web Token)** based authentication instead of simple Client ID/Secret key validation to secure API access and ensure that only authorized Africom client applications can submit notification commands.

4.8 Recommendations

- Based on the experience gained and the identified opportunities for improvement, the following recommendations are provided for the internship host company and the university:

4.8.1 Recommendations for Africom Technologies PLC

- **Introduce Formal Client Integration:** We recommend that future interns be given the opportunity to participate in the final integration phase—connecting the developed service (like the PNS) with an actual, live client application. This experience in external API consumption, testing in a staging environment, and final deployment is crucial for understanding the complete software development lifecycle.
- **Extended Technical Training:** To maximize the intern's contribution time, we suggest structuring a **brief, mandatory training session** (1-2 days) at the start of the internship covering the specific architectural patterns (e.g., CQRS, Repository Pattern) and framework conventions (e.g., Dependency Injection) that will be used in the assigned project.

4.8.2 Recommendations for Debre Berhan University

- **Extend Internship Duration:** We strongly recommend extending the required internship duration from two months to a minimum of **three or four months**. This extended time is necessary for students to move beyond the initial phase of environment setup and basic development into advanced topics like system optimization, security hardening, and performance testing.
- **Incorporate Enterprise Architectures:** The curriculum should place a greater practical emphasis on contemporary architectural patterns, such as **CQRS/Mediator**, **Microservices**, and **Layered Architecture**. This will better align the theoretical knowledge with the high technical standards expected in the industry, minimizing the initial knowledge gap faced by interns.

Part Five: General Conclusion and Recommendation

5.1. General Conclusion

- The internship at Africom Technologies PLC, focused on developing the **Component-Based Push Notification System (PNS)**, proved to be an invaluable and essential experience for my professional development. It successfully fulfilled the university's requirement to bridge the gap between abstract academic theories and challenging real-world software engineering practices.

- The work provided intensive, hands-on exposure to an enterprise technology stack, notably the **C#/.NET Core** framework, **Microsoft SQL Server**, and professional version control via **GitHub**. Through direct involvement in designing the API, integrating the email service, and implementing the critical **email open tracking** mechanism, my technical skills were significantly accelerated. Crucially, the mandatory use of advanced design patterns, such as the **CQRS (Command Query Responsibility Segregation)** and the **Mediator pattern**, provided a practical understanding of how to build systems that are not only functional but also scalable, modular, and maintainable.

- Beyond the technical skills, the internship fostered a robust professional character. I significantly improved my **team-playing skills** through mandatory peer code reviews and collaborative debugging, strengthened my **leadership skills** by guiding peers through complex GitHub conflicts, and gained a deep appreciation for the **work ethics** required in a successful IT firm, particularly punctuality and disciplined reporting. Overall, the experience was a profound step toward establishing a career in information systems, equipping me with the confidence and practical expertise required to tackle future career challenges.

References

- *(Note on Formatting: The DBU guideline recommends using either IEEE or APA. I have formatted the list below using **APA 7th Edition** style for consistency and professional presentation, as it is widely used in Information Systems documentation.)*
- **Africom Technologies PLC internal documentation and training materials.** (2024). *Proprietary Internal Guides and Project Documentation*. Africom Technologies PLC.
- Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Addison-Wesley Professional. (Used for theoretical knowledge upgrades on architecture).
- Microsoft Documentation. (n.d.). **ASP.NET Core Fundamentals**. Retrieved from [Insert Specific MSDN URL if available, otherwise general reference remains].
- Microsoft. (n.d.). **Microsoft SQL Server Documentation**. Retrieved from [Insert Specific MSDN URL if available].
- **Swagger API Documentation: OpenAPI Specification**. (n.d.). Retrieved from [Insert Specific Swagger URL if available].
- **W3Schools Tutorials: C#, SQL, and Web API basics**. (n.d.). Retrieved from [Insert Specific W3Schools URL if available].
- **Stack Overflow discussions and community resources for debugging.** (n.d.). [General reference for community-based technical solutions].

Appendices

(Note: Appendices are typically unnumbered and use Roman numerals or capital letters for internal pagination.)

Appendix A: Sample Code Snippets

- **A.1:** Code from EmailService.cs showing the **SMTP configuration structure** and the logic for **tracking pixel injection**.
- **A.2:** Code from NotificationController.cs showing the use of the **IMediator** interface to send Commands and Queries
- **A.3:** Snippet from appsettings.json showing the **SmtpSettings** configuration (Host, Port, Username/App Password, etc.) and the **ConnectionStrings** for the database.

Appendix B: Screenshots

- **B.1:** Screenshot of the PNS API endpoints as displayed in **Swagger UI** (showing the POST /Notification and GET /Notification/{id}/track endpoints).
- **B.2:** Screenshot of a **test email** received by the recipient, confirming successful delivery.
- **B.3:** Screenshot of the SQL Server Management Studio (SSMS) table view showing data entry into the **Notifications** and **Client** tables.

Appendix C: Database Schema Diagram

- **C.1:** The complete **Entity Relationship Diagram (ERD)** for the PushNotificationDb, illustrating the relationships between Clients, Notifications, and Tracking Logs.

Appendix D: Project Workflow / Data Flow Diagram

- **D.1:** A detailed **Data Flow Diagram (DFD)** illustrating the internal flow of a notification request: Controller → Mediator → Command Handler → Repository → Database/Email Service.

Table 4.2 Functional Requirements of PNS

No	Functional Requirement	Description

1	Manage Client Applications	Allow administrators to register, update, and deactivate client applications.
2	Send Notifications	Enable client applications to send notification requests through the RESTful API.
3	Manage Notification Types	Support multiple notification channels such as Email, SMS, and internal alerts.
4	Track Email Opens	Log when an email notification is opened using an embedded tracking pixel.
5	Retrieve Notification History	Allow retrieval of past notifications and their status (sent, opened, failed).
6	Prioritize Notifications	Handle notification priorities (High, Medium, Low) and process accordingly.
7	Authenticate API Access	Verify client credentials (ClientID and SecretKey) before allowing access.
8	Log System Activity	Log all actions such as send, open, and failure events for auditing.
9	Provide API Documentation	Expose Swagger UI for endpoint documentation and testing.

Table 4.3 Non-Functional Requirements of PNS

No	Non-Functional Requirement	Description
1	Performance	Process at least 100 notification requests per second under normal load.
2	Scalability	Support horizontal scaling to handle increased client traffic.
3	Reliability	Achieve 99% uptime and ensure reliable message delivery.
4	Security	Secure API endpoints with HTTPS and require valid client credentials.
5	Maintainability	Follow layered architecture and CQRS pattern for easy maintenance.
6	Usability	Provide self-documented Swagger API and clear response messages.
7	Portability	Be deployable on both on-premises and cloud (Azure, AWS) environments.
8	Availability	Ensure continuous availability during business hours with minimal downtime.

9	Fault Tolerance	Gracefully handle SMTP failures and retry sending where applicable.
10	Auditability	Log all transactions for later review and analysis.